

Semantic Reduction Is the Bottleneck: A Three-Layer Reliability Framework for AI for Mathematics

Elite 20 Program

AI4Math Reliability Study Group

September 2026

Abstract

Large language models can now produce mathematical proofs that read as expert work, and they can also produce proofs that are subtly and irreparably wrong. The reliability question for AI for Mathematics (AI4Math) is therefore not “can the model solve the problem?” but “when the model says it has solved the problem, how much should we believe it?” Current answers fall into two families: *post-hoc verification*, which checks a finished natural-language proof with an external tool, and *end-to-end formalisation*, which asks the model to emit a proof directly in a proof assistant such as Lean. We argue that both families mislocate the difficulty, and we place the failure mode in a specific architectural position.

We propose a three-layer decomposition of AI4Math systems—*generation*, *semantic reduction*, and *verification*—and argue that the semantic reduction layer, which maps informal mathematical claims onto typed formal objects, is the dominant bottleneck. We formalise this claim by introducing a typed intermediate representation (SRIR) with an explicit anchoring map, defining a *verifiability score* $\mathcal{V}(x)$ that separates anchored from unanchored fragments of a derivation, and giving a taxonomy of seven reduction defects. We then present a falsifiable prediction: the *narrowing hypothesis*, that a system’s error rate at fixed solution effort is bounded by its unanchored fraction. Finally we give a reusable *Reliability Checklist* for auditing AI4Math claims, and we state the boundary conditions under which our analysis does and does not apply.

The contribution is conceptual and methodological rather than empirical: we do not report large-scale experiments, and we are explicit that the framework is motivated by, and consistent with, published results rather than proved by them. Our aim is to give researchers and practitioners a vocabulary precise enough to locate reliability failures, and a checklist precise enough to act on them.

Keywords: AI for Mathematics, autoformalisation, proof assistants, neuro-symbolic reasoning, reliability, verifiability, hallucination.

1 Introduction

1.1 The reliability gap

Between 2022 and 2026, machine mathematical reasoning moved from word problems to research-adjacent results. Systems trained with reinforcement learning against a proof assistant reached medal-level performance on olympiad problems [9, 3]; the same neuro-symbolic recipe rediscovered algorithms that had resisted human search for decades [6, 13]; and language models fine-tuned on mathematical corpora began to produce fluent, confident, multi-step arguments [1].

Fluency, however, is not correctness. A model that has learned the *register* of mathematical prose has learned to produce text that looks like a proof, and the surface features that distinguish a valid proof from an invalid one—whether a limit interchange is licensed, whether an induction is

well-founded, whether a cited lemma’s hypotheses are actually met—are exactly the features that a next-token predictor has no direct incentive to preserve. Empirical work has repeatedly found that apparent mathematical competence degrades sharply once the task requires genuine multi-step composition rather than pattern retrieval [7]. The gap between “produces a proof-looking object” and “produces a proof” is the reliability gap.

1.2 Two answers, and why they are incomplete

The field’s response has been to add a verifier.

Post-hoc verification takes the model’s natural-language proof and subjects it to an external check: a symbolic engine, an SMT solver, or a critic model. This is attractive because it is cheap and does not constrain the generator. It is also structurally weak: if the natural-language derivation contains an unstated inference, a symbol the checker cannot type, or an implicit appeal to “standard facts,” then the checker either fails on a correct proof or—worse—succeeds on an incorrect one by checking a weaker formalisation than the author intended.

End-to-end formalisation bypasses this by asking the model to emit a proof directly in a proof assistant. When it succeeds, the resulting artifact is checkable by a small trusted kernel and its correctness is as certain as the foundations allow [5, 14]. This is the strongest available notion of reliability, and it is why the strongest systems use it. But it does not remove the difficulty; it relocates it. The model must now produce a *formal* object, and the mapping from the informal statement a human asked about to the formal statement the kernel checks is performed by the model itself, in natural language, before any verification happens. If that mapping is wrong, the kernel will faithfully certify a theorem nobody asked about.

1.3 Thesis

We argue that both answers share a blind spot: they treat the transition from informal mathematics to formal mathematics as an implementation detail. We claim it is the central object of study. Concretely:

Thesis. In a layered AI4Math system, the dominant source of reliability failure is neither the generator nor the checker, but the *semantic reduction layer* that maps informal mathematical content onto typed formal objects. Reliability therefore cannot be obtained by improving the final check alone; it requires making the reduction layer an explicit, instrumented, and auditable system component.

This is a claim about where difficulty lives, and it is falsifiable in the sense that a system could be exhibited whose reduction is demonstrably faithful and whose residual errors lie elsewhere. We state the conditions under which we expect the thesis to fail in Section 9.2.

1.4 Contributions

1. **A three-layer decomposition** of AI4Math systems (Section 3) that separates generation, semantic reduction, and verification, and that makes the interfaces between them explicit and *instrumentable*.
2. **A typed intermediate representation (SRIR)** with a formal anchoring map *Anchor* and a notion of *semantic closure* (Section 4), designed so that every node of a derivation is either anchored to a checkable object or explicitly marked as unanchored.
3. **A verifiability measure $\mathcal{V}(x)$** (Section 5) that quantifies the anchored fraction of a derivation, together with a taxonomy of seven reduction defects (Section 6) that gives failures names.

4. **The narrowing hypothesis** (Section 7): a falsifiable, testable statement connecting the unanchored fraction of a derivation to residual error rate, with a proposed experimental design and explicit confounders.
5. **A Reliability Checklist** (Section 8 and Appendix A) for auditing AI4Math claims in practice.

1.5 Scope and non-claims

This paper is a position and framework paper. We do not report new experiments, we do not claim superiority over existing systems, and we do not claim that the semantic reduction layer is the *only* source of unreliability. Sections 9.2 and 9 state the limits of the analysis explicitly. Where we cite empirical numbers, they are from the cited primary sources and are used to motivate the framework, not to validate it.

2 Background and Related Work

2.1 Proof assistants and the trusted kernel

Interactive theorem provers (ITPs) such as Coq, Isabelle, and Lean rest on a small trusted kernel that checks each inference against a fixed set of rules [12]. The Lean prover and its mathematical library, `mathlib`, provide a modern instantiation with a large, machine-readable body of formalised mathematics [5, 14]. The architectural property that matters for us is the *de Bruijn criterion*: all proof construction, however clever or however unreliable, is reduced to a check against a small kernel. This makes the kernel an *absolute* oracle for the formal claim it is given—and a silent bystander to the question of whether that formal claim is the intended one.

That gap—between the intended claim and the formalised claim—is the space our framework is designed to make visible. We call it the *anchoring gap*.

2.2 Autoformalisation

Autoformalisation is the task of translating informal mathematical statements and proofs into formal ones. Wu et al. [16] established that large language models can perform this translation with useful but imperfect accuracy, reporting that a substantial fraction of translated statements are syntactically well-formed yet semantically divergent from their sources. Zhou et al. [19] turn this into a reliability mechanism: formalise the model’s own reasoning and let the checker veto it, finding that verification-by-autoformalisation outperforms majority voting on grade-school arithmetic.

This line is the closest to our position, and our contribution is complementary rather than competing. Prior work treats autoformalisation as a *step* whose output may be filtered. We treat it as a *layer* whose internal structure determines the ceiling on achievable reliability, and we provide a representation and a metric that make the layer’s residual unreliability measurable rather than merely detectable in aggregate.

2.3 Neural-symbolic theorem proving

The strongest current systems pair a learned policy with a proof assistant. AlphaProof formalises olympiad problems, searches with a learned value function and tree search, and certifies every successful proof in Lean; it reached silver-medal level at IMO 2024 [9]. AlphaGeometry2 similarly attains gold-medal-level geometry performance using a dedicated domain representation and a symbolic deduction engine [3]. Open systems such as Goedel-Prover and retrieval-augmented provers such as LeanDojo push these capabilities into reproducible, open-weight territory [11, 18]. Yang et al. survey the resulting research programme [17].

These systems demonstrate that the neuro-symbolic loop works. They also illustrate our thesis in their own deployment reports: capability is bounded by the coverage of the formal library and by the fidelity of the informal-to-formal translation, not by the search procedure. Geometry required a separate system precisely because the shared formal library did not cover olympiad-style geometry well enough to make reduction reliable [3]. That is a reduction-layer failure dictating an architectural workaround.

2.4 Language models for mathematics and step-level verification

Llemma demonstrates that continued pretraining on mathematical and code corpora yields a substantially stronger informal reasoner [1]. Chain-of-thought prompting elicits multi-step derivations by making intermediate steps explicit [15]. Step-level supervision goes further, rewarding correct *process* rather than only correct outcomes, and producing models that can localise their own errors [10]. The relevance to us is direct: step-level verification presupposes that steps are *identifiable objects* with checkable content, which is exactly what the SRIR representation of Section 4 provides, and exactly what raw natural-language derivations lack.

2.5 Benchmarks and what they measure

GSM8K and MATH measure final-answer correctness on word problems and competition mathematics respectively [4, 8]. DeepMath-Creative measures creativity along three axes—new concepts, new methods, and new examples—and finds that current models struggle most on tasks requiring genuine construction [2]. Frieder et al. document sharp degradation of apparent mathematical competence under careful evaluation [7].

The methodological point we take from this literature is that *answer-level* benchmarks cannot distinguish a model that reasoned correctly from one that arrived at a correct answer through invalid steps. Our $\mathcal{V}(x)$ measure is an attempt to supply the missing axis: not whether the answer is right, but how much of the derivation is anchored.

2.6 What is missing

Across this literature the reduction step is variously treated as a preprocessing detail, a filtering mechanism, or an implicit competence. We are not aware of a framework that (i) gives informal-to-formal reduction its own architectural layer with defined interfaces, (ii) assigns each derivation node an explicit anchoring status, and (iii) yields a scalar measure that can be tracked across systems. Providing these is the contribution of this paper.

3 A Three-Layer Decomposition

3.1 The layers

We decompose any AI4Math system that aspires to verifiable output into three layers with typed interfaces.

Definition 3.1 (Three-layer decomposition). An AI4Math system is a tuple $\langle G, R, C \rangle$ where:

- $G : \mathcal{N} \rightarrow \mathcal{N}$ is the **generation layer**, mapping an informal problem statement to an informal derivation;
- $R : \mathcal{N} \rightarrow \mathcal{I}$ is the **semantic reduction layer**, mapping informal mathematics to typed formal objects in an intermediate representation $\mathcal{I}$ (Section 4);
- $C : \mathcal{I} \rightarrow \{\top, \perp\}$ is the **verification layer**, deciding whether a formal object is accepted by the target formal system $\mathcal{F}$.

The system’s output on input p is $C(R(G(p)))$.

The composition in Definition 3.1 is deliberately simple; its point is to expose that a single user-visible verdict is the product of three independently unreliable components.

3.2 Why the middle layer is where failures concentrate

Each layer has a characteristic failure mode.

Layer 1 (generation) fails by producing wrong content. This failure is *loud*: the derivation is wrong, and any downstream check that has enough information can catch it. Crucially, G operates in $\mathcal{N}$, where the notion of “wrong” is itself contested.

Layer 3 (verification) fails by being incomplete. A kernel may reject a valid proof (incompleteness, resource limits) but, by the de Bruijn criterion, it does not accept invalid proofs. Its failure mode is conservative and detectable: rejection.

Layer 2 (semantic reduction) fails by being *silently unfaithful*. The reduction may map an informal claim to a formal claim that is checkable, syntactically well-formed, *and not the claim the user meant*. Every downstream component then behaves correctly with respect to the wrong target. This is the only failure mode in the system that is simultaneously (i) undetectable by the verifier, (ii) indistinguishable from success at the user interface, and (iii) able to corrupt a proof that is internally flawless.

Proposition 3.1 (Asymmetry of detectability). *Let C satisfy the de Bruijn criterion. Then:*

1. *Every error introduced at Layer 3 is either detectable as a rejection or absent.*
2. *Errors introduced at Layer 2 are not detectable by C alone whenever the reduced object is internally consistent.*

Proof sketch. (1) holds by definition of the de Bruijn criterion: C accepts only objects derivable in $\mathcal{F}$, so an invalid object yields $\perp$. (2) Suppose R maps claim φ to formal object $i \neq R(\varphi)$ but i is derivable in $\mathcal{F}$; then $C(i) = \top$ while the intended claim is unverified. No property of C alone can distinguish this case from a faithful reduction, since C has access only to i and not to φ . □ □

Proposition 3.1 is the formal core of our thesis. Note what it does *not* say: it does not claim Layer 2 errors are the most frequent, nor that Layers 1 and 3 are unimportant. It claims that Layer 2 errors are uniquely invisible to the verification layer, and therefore cannot be addressed by strengthening verification. This is a statement about *where additional verification effort has no leverage*, which is precisely the kind of statement that should guide architecture.

3.3 The pipeline as a contract

Read left to right, Figure 1 describes a pipeline. Read as a contract, it imposes obligations. Layer 1 owes Layer 2 a derivation whose steps are *individually identifiable*. Layer 2 owes Layer 3 formal objects that are *type-correct and semantically anchored*. Layer 3 owes the system a verdict on *exactly the object it was given*. Most current systems satisfy the third obligation rigorously and the first two informally—which is consistent with the empirical pattern that verification is reliable and reliability overall is not.

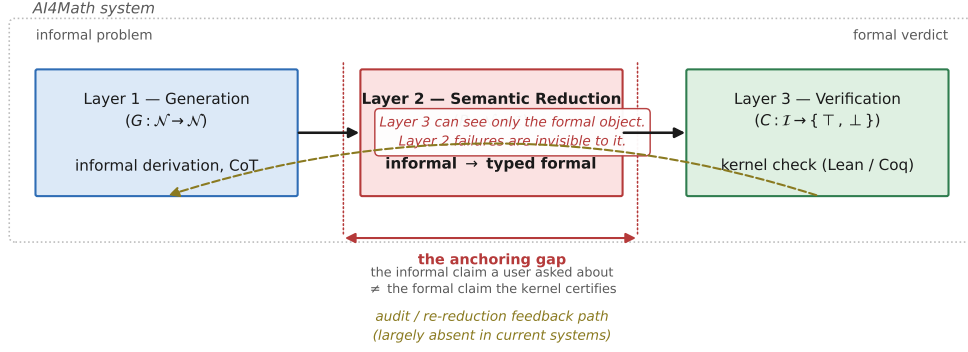


Figure 1: The three-layer decomposition. Solid arrows are the data path; the dashed arrow is the feedback path that current systems largely omit. The *anchoring gap* is the distance between the informal claim a user asked about and the formal claim the kernel certifies. Layer 2 is the only layer whose failures are invisible to Layer 3 (Proposition 3.1).

4 SRIR: A Typed Intermediate Representation

4.1 Design goals

We want a representation that makes the anchoring gap *measurable*. Four requirements follow directly from the failure analysis above.

1. **Step identity.** Every inference step must be a first-class object with a stable identifier, so that anchoring can be assessed per step rather than per proof.
2. **Explicit typing.** Every object must carry a type, so that type mismatch—a common silent failure—becomes a checkable condition rather than a plausibility judgement.
3. **Explicit dependencies.** Each step must declare which prior steps and which library facts it consumes, so that an unanchored dependency cannot hide behind a fluent sentence.
4. **Explicit unanchored status.** A step that cannot be anchored must be *representable as unanchored*. The representation must not force a choice between “anchored” and “absent.”

Requirement 4 is the one that distinguishes SRIR from a proof object. A proof object records what was proved; SRIR records what was *claimed* and how much of it was *secured*.

4.2 Definition

Definition 4.1 (SRIR graph). An SRIR document is a finite directed acyclic graph $\mathcal{G} = (V, E, \tau, \delta, \alpha)$ where:

- V is a finite set of *derivation nodes*, the steps;
- $E \subseteq V \times V$ encodes dependency: $(u, v) \in E$ means v consumes the result of u ;
- $\tau: V \rightarrow \mathcal{T}$ assigns each node a type drawn from a type system $\mathcal{T}$ that includes at least Prop, Term, Assumption, LemmaRef, and Goal;

- $\delta : V \rightarrow \mathcal{F}$ assigns each node a formal counterpart in the target system $\mathcal{F}$ (a term, a proof obligation, or a goal state);
- $\alpha : V \rightarrow \{\mathbf{A}, \mathbf{U}\}$ is the *anchoring status*: $\alpha(v) = \mathbf{A}$ (*anchored*) iff $\delta(v)$ is well-typed, references only library facts that exist in $\mathcal{F}$, and is accepted by the kernel when its dependencies are discharged; $\alpha(v) = \mathbf{U}$ (*unanchored*) otherwise.

The *anchored subgraph* $\mathcal{G}_A = (V_A, E_A)$ is the restriction of $\mathcal{G}$ to $V_A = \{v : \alpha(v) = \mathbf{A}\}$.

Definition 4.2 (Semantic closure). $\mathcal{G}$ is *semantically closed* if $\alpha(v) = \mathbf{A}$ for the root node and every node in the transitive dependency cone of the root.

Semantic closure is the property a system should aim for: not merely that the final answer is certified, but that the certification does not rest on any unanchored step.

4.3 The anchoring map and the anchoring gap

Definition 4.3 (Anchoring map). The anchoring map $\text{Anchor} : \mathcal{N} \rightarrow \mathcal{I}$ is the concrete realisation of the semantic reduction layer R : it sends an informal mathematical claim to an SRIR node whose formal counterpart is $\delta(v)$. The *anchoring gap* of a reduction is the pair $(\varphi, \delta(\text{Anchor}(\varphi)))$ —the informal claim and the formal object that will actually be checked in its place.

The anchoring gap is not a defect. It is unavoidable: informal mathematics runs on shared understanding, and any formalisation must choose a specific rendering. The question that matters is whether the gap is *disclosed*. A system that emits a Lean file and a natural-language gloss, with no correspondence between them, has an undisclosed gap; a system that emits an SRIR graph in which every node carries its informal gloss, its type, and its anchoring status has a disclosed one. Disclosure is what makes auditing possible.

Example 4.1 (A silent reduction failure). A user asks: “*Show that every continuous function on $[0, 1]$ attains a maximum.*” A reduction may render this as a statement about *sequentially* continuous functions on a *compact metric space*. In Lean’s type theory, with the appropriate instances, this is derivable—and the kernel will accept the proof. The formal object is correct. But the informal claim, in a context where “continuous” was meant in the general topological sense and the space was assumed merely bounded, has not been established. Layer 3 reports success; Layer 2 has silently substituted a friendlier claim. Under SRIR, the **Assumption** node introducing the compactness instance carries $\alpha = \mathbf{U}$, the unanchored status propagates up the dependency cone, and semantic closure (Definition 4.2) fails—a fact visible at the interface rather than hidden behind a green check.

Example 4.1 is the archetype. It is worth noting that no amount of improvement to Layer 3 addresses it, which is the practical content of Proposition 3.1.

4.4 Construction: the reduction algorithm

Algorithm 1 gives a reference construction of an SRIR graph. It is deliberately stated at the level of an interface rather than an implementation: any concrete reduction method—LLM-based autoformalisation, retrieval over a library, or a hybrid—can be plugged in as the subroutine **PROPOSEFORMAL**. The contribution is the surrounding instrumentation, not the proposal mechanism.

Two design decisions in Algorithm 1 deserve emphasis.

Line 7 is where the difficulty lives. **PROPOSEFORMAL** is the irreducibly hard part, and it is the part that receives the least engineering attention in current systems. We do not solve it; we make its failures attributable.

Lines 14–19 never discard a node. A step that cannot be anchored is retained with $\alpha = \mathbf{U}$. This is the technical manifestation of requirement 4: the representation must be able to say “I do not know” without either failing or lying.

Algorithm 1 REDUCETOSRIR — instrumented semantic reduction

Require: informal derivation $d \in \mathcal{N}$; formal system $\mathcal{F}$; library $\mathcal{L}$; prover oracle CHECK

Ensure: SRIR graph $\mathcal{G} = (V, E, \tau, \delta, \alpha)$

```
1:  $V \leftarrow \emptyset$ ;  $E \leftarrow \emptyset$ 
2:  $S \leftarrow \text{SEGMENT}(d)$  ▷ split into individually identifiable steps
3: for all  $s \in S$  do
4:    $i \leftarrow \text{PROPOSEFORMAL}(s)$  ▷ LLM / retrieval / hybrid: informal step to formal object
5:    $t \leftarrow \text{INFERTYPE}(i)$  ▷ type synthesis over  $\mathcal{T}$ 
6:    $D \leftarrow \text{RESOLVEDEPS}(i, V, \mathcal{L})$  ▷ which prior nodes and library facts does  $i$  need?
7:    $v \leftarrow \text{NEWNODE}(i, t, D)$ ;  $V \leftarrow V \cup \{v\}$ 
8:    $E \leftarrow E \cup \{(u, v) : u \in D\}$ 
9: end for
10: anchor pass (bottom-up over topological order of  $\mathcal{G}$ )
11: for all  $v \in \text{TOPOORDER}(\mathcal{G})$  do
12:   if  $\text{WELLTYPED}(\delta(v))$  and  $\text{DEPSEXIST}(v, \mathcal{L})$  and  $\text{CHECK}(\delta(v)) = \top$  then
13:      $\alpha(v) \leftarrow \mathbf{A}$ 
14:   else
15:      $\alpha(v) \leftarrow \mathbf{U}$  ▷ never silently drop the node
16:   end if
17: end for
18: return  $\mathcal{G}$ 
```

5 Measuring Verifiability

5.1 The verifiability score

Semantic closure is binary, but in practice a derivation is partially anchored, and we want to track partial progress. We therefore define a graded measure. Because steps differ in importance—a missing library reference in a routine algebraic manipulation is not the same as an unanchored induction hypothesis—we weight nodes by their contribution to the dependency cone of the root.

Definition 5.1 (Verifiability score). Let $\mathcal{G} = (V, E, \tau, \delta, \alpha)$ be an SRIR graph with root r . Let $w : V \rightarrow \mathbb{R}_{>0}$ be a weighting. The *verifiability score* is

$$\mathcal{V}(\mathcal{G}) = \frac{\sum_{v \in V} w(v) \mathbb{I}[\alpha(v) = \mathbf{A}]}{\sum_{v \in V} w(v)} \in [0, 1].$$

A natural default is the cone weight, which measures how much of the proof *depends* on each node:

$$w_{\text{cone}}(v) = |\{u \in V : v \text{ is an ancestor of } u\}|.$$

Proposition 5.1 (Basic properties). *For any SRIR graph $\mathcal{G}$ with root r :*

1. $\mathcal{V}(\mathcal{G}) = 1 \iff$ every node is anchored.
2. If $\mathcal{G}$ is semantically closed then $\mathcal{V}(\mathcal{G})$ restricted to the dependency cone of r equals 1.
3. $\mathcal{V}$ is invariant under relabelling of nodes and monotone in the following sense: if $\alpha_1 \leq \alpha_2$ pointwise (with $\mathbf{U} < \mathbf{A}$) then $\mathcal{V}_{\alpha_1} \leq \mathcal{V}_{\alpha_2}$.

Proof. (1) and (3) are immediate from the definition, since the indicator is pointwise monotone and the weights are positive. For (2): semantic closure (Definition 4.2) requires $\alpha = \mathbf{A}$ on the cone of r ; restricting the sum to that cone makes every term in the numerator contribute, giving ratio 1. $\square$

Table 1: Taxonomy of semantic reduction defects. The *visibility* column records whether the defect is detectable by the verification layer alone; every entry marked *invisible* is a manifestation of Proposition 3.1.

ID	Defect	Description	Visible to Layer 3?
D1	Type coercion	Informal object silently retyped to fit a convenient formal type (e.g. “number” $\rightarrow$ Nat).	No
D2	Quantifier drift	Scope or strength of a quantifier changes in translation (e.g. $\forall$ weakened to $\exists$, or a bound relaxed).	No
D3	Hidden hypothesis	A condition used in the informal argument is not discharged as an explicit premise in the formal object.	No
D4	Lemma mismatch	A cited result is applied outside its stated hypotheses, or a similarly-named library fact is substituted.	No
D5	Boundary loss	Edge cases (degenerate, empty, limiting) present in the informal claim are absent from the formal one.	No
D6	Notational collision	The same informal symbol denotes distinct formal objects (or vice versa) with no disambiguation.	No
D7	Unanchored assertion	A step is formalised as an unproved assumption presented as established.	Partial*

* D7 becomes visible only when the assumption is made explicit in the SRIR graph; in a plain natural-language proof it is indistinguishable from a step.

Property (3) is the one that makes $\mathcal{V}$ useful operationally: anchoring an additional step can never decrease the score. A system can therefore be improved incrementally, and the improvement is measurable.

5.2 Why not accuracy, and why not pass rate

It is worth stating why we do not simply use accuracy or formal pass rate.

Consider two systems, both scoring 0.70 on a benchmark. System A produces seven fully anchored proofs and three fluent but unfounded ones. System B produces ten proofs of which seven are anchored, and the three failures are anchored on all but one critical step. Accuracy cannot distinguish them. $\mathcal{V}$ can: System B’s failures are *localised* and therefore repairable, System A’s are not. For any downstream consumer—a mathematician deciding whether to trust a result, or an engineer deciding where to invest effort—this distinction is the whole question.

5.3 Relation to step-level supervision

$\mathcal{V}$ is a *static* property of a completed derivation, whereas process supervision [10] is a *training signal* over steps. The two are complementary: process supervision produces models whose steps are individually more likely to be valid, and $\mathcal{V}$ measures whether those steps have been securely connected to a formal foundation. A model can have high step-validity and low $\mathcal{V}$ if its steps are plausible but unanchored—a combination we conjecture is common in current systems and which current benchmarks cannot detect.

6 A Taxonomy of Reduction Defects

A measure is only actionable if failures can be named. We propose a taxonomy of seven defects at the semantic reduction layer, organised by the kind of distortion they introduce. The taxonomy is intended to be jointly exhaustive enough for auditing purposes and mutually distinguishable enough to be annotatable.

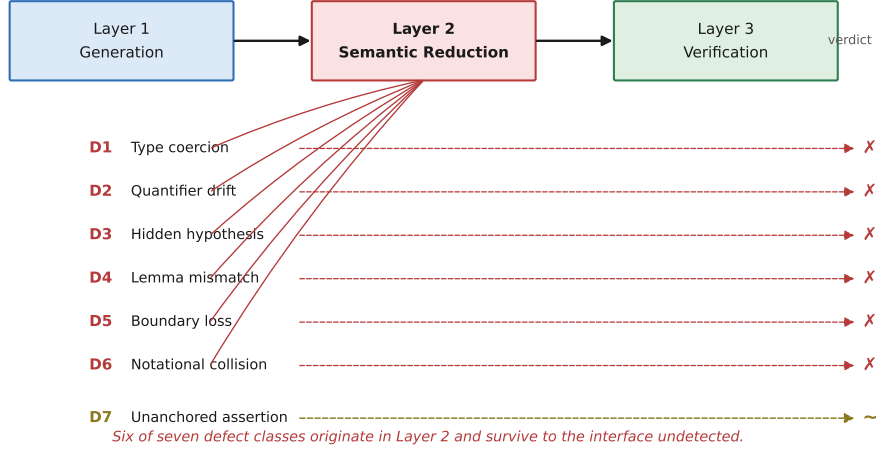


Figure 2: Where the seven defect classes originate in the pipeline. Defects D1–D6 arise in the mapping performed by Layer 2 and survive to the interface undetected. D7 is partially visible because SRIR forces the assumption to be represented explicitly rather than absorbed into prose.

Table 1 is the concrete deliverable of this section. Its purpose is diagnostic: given a failed AI4Math claim, an auditor can ask which of D1–D7 occurred, and the answer determines the remedy. D1 and D6 suggest tightening the type system. D2, D3, and D5 suggest an explicit statement-diffing pass between informal and formal claims. D4 suggests verifying library citations against their actual hypotheses rather than their names.

The final column is a restatement of our central claim in tabular form: six of seven defect classes are invisible to the verification layer. This is why reliability cannot be obtained by making the checker better.

7 The Narrowing Hypothesis

We now state the central empirical claim of the framework. It is offered as a falsifiable hypothesis, not a result.

Narrowing hypothesis. For an AI4Math system $\langle G, R, C \rangle$ and a benchmark $\mathcal{B}$, the rate of *accepted-but-wrong* outputs is bounded above by a function of the unanchored fraction $1 - \mathcal{V}$ of the derivations the system accepts:

$$\Pr_{\substack{x \sim \mathcal{B} \\ C(G(x)) = \top}} [x \text{ is actually wrong}] \lesssim f(1 - \mathcal{V}(\mathcal{G}_x))$$

for some nondecreasing f with $f(0) = 0$.

The hypothesis says: *a system whose accepted derivations are fully anchored cannot be accepted-but-wrong, and as anchoring degrades, the rate of accepted-but-wrong outputs rises.* The first half is essentially a restatement of Proposition 3.1 under the assumption that anchoring is sound; the content is in the second half, which asserts a quantitative relationship that could fail.

7.1 Designing a test

The hypothesis is testable with equipment that already exists. We sketch a design and name its weaknesses, because a framework that cannot say how it would be refuted is not doing useful work.

1. **Construct derivations with controlled anchoring.** Take a set of problems with known formalisations. Generate derivations, then deliberately degrade them by injecting each defect class D1–D7 at a controlled rate, producing SRIR graphs with known $\mathcal{V}$.
2. **Measure accepted-but-wrong rate.** For each derivation, run the verification layer and record acceptance. Where acceptance occurs, determine ground truth by comparing the formalised claim against the intended claim by human adjudication.
3. **Fit and test.** Estimate f and test whether the accepted-but-wrong rate is bounded by it. The hypothesis fails if the rate stays high at $\mathcal{V} \rightarrow 1$, or is insensitive to $1 - \mathcal{V}$.

7.2 Confounders and threats to validity

We state these explicitly, in the spirit of the boundary conditions requested of any framework claim.

- **Adjudication cost.** Determining “actually wrong” requires comparing intended and formalised claims—the very anchoring gap we are studying. A cheap proxy is to use hand-formalised reference claims; this restricts the test to problems where a reference exists and biases toward well-formalised domains.
- **Degradation realism.** Injected defects may not resemble the defects a real model produces. A model’s reduction errors are distributionally different from random injections, plausibly because they cluster in the directions the model finds convenient.
- **Weighting sensitivity.** $\mathcal{V}$ depends on the choice of w . Results should be reported under at least two weightings, e.g. the uniform and cone weightings, and the conclusion claimed only if they agree.
- **Closed-world bias.** Benchmarks drawn from well-formalised libraries have unusually reliable reduction because the informal and formal languages are close. Performance outside such domains is the case we are most interested in and the hardest to test.

7.3 What the hypothesis is not

It is not a claim that reducing unanchored steps is sufficient for reliability: a fully anchored derivation of the wrong theorem is still the wrong theorem, though the framing of “wrong theorem” is then resolved by the type system rather than by a judgement call. Nor is it a claim that $\mathcal{V}$ is the only useful measure; it is offered as a candidate axis that current benchmarks lack.

8 A Reliability Checklist for AI4Math Claims

The framework’s practical output is a checklist. It is designed for a practitioner facing a claim of the form “an AI system has established mathematical result X ” and asking how much credence to extend. The full form appears in Appendix A; we state the six questions here with their rationale.

1. **What exactly was formalised?** Obtain the formal statement, not only the informal gloss. If no formal statement exists, ask what the strongest available anchor is (a symbolic check, a numerical verification, an independent replication).
2. **Is the informal-to-formal mapping disclosed step by step?** If the system reports only a final kernel verdict, the anchoring gap is undisclosed and D1–D6 are unauditable.

3. **Which steps are anchored, and which are unanchored?** In SRIR terms, what is $\mathcal{V}$, and which nodes carry $\alpha = \mathcal{U}$?
4. **Are the unanchored steps load-bearing?** A single unanchored step on the critical path matters more than several off it; this is what the cone weighting is for.
5. **Do the library citations hold up?** Check that cited lemmas are applied within their stated hypotheses (D4), rather than matched by name.
6. **What is the boundary of the claim?** Which domains, library coverage assumptions, and problem classes does the result depend on? A result established in a well-formalised closed domain does not transfer to an open one.

The checklist is intentionally ordered by cost: questions 1–2 are answerable from system outputs alone, questions 3–5 require instrumentation of the kind SRIR provides, and question 6 is a judgement that no tool supplies.

9 Discussion

9.1 Implications for system design

If the analysis is right, three design consequences follow.

First, *the reduction layer deserves its own engineering effort and its own evaluation*. Current systems invest in generation (better models) and verification (better kernels and search) and treat reduction as glue. Our analysis says the glue is load-bearing.

Second, *disclosing the anchoring gap is more valuable than closing it*. Closing it entirely is likely impossible for informal mathematics, which depends on shared understanding that resists formalisation. Disclosing it is a representation and interface problem, and our SRIR proposal is one concrete way to do it.

Third, *benchmarks should report verifiability alongside accuracy*. A leaderboard that reports only final-answer correctness cannot distinguish a system that reasoned soundly from one that got lucky, nor can it detect the regression that occurs when a system becomes fluent faster than it becomes sound.

9.2 Limitations

We state plainly what this framework does not establish.

1. **No empirical validation.** We present no experiments. The narrowing hypothesis is a proposal, and the taxonomy is a design artifact, not a validated instrument. Our citations to empirical work motivate the framework; they do not demonstrate it.
2. **Annotation cost is unaddressed.** Producing SRIR graphs requires segmentation and per-step formalisation, both of which are themselves unreliable and potentially expensive. We have not measured this cost, and it may be prohibitive for long derivations.
3. **The taxonomy is not exhaustive.** D1–D7 are drawn from the failure modes reported in the cited literature and our own analysis of worked examples. Other defect classes surely exist; we expect the list to be extended, and we regard it as a snapshot rather than a complete classification.
4. **Semantic closure is strong.** Requiring every node in the cone of the root to be anchored is demanding, and may be unattainable for derivations that legitimately rely on standard results not yet in a library. The measure $\mathcal{V}$ exists partly to accommodate this, but the framework is likely most useful for well-formalised domains.

5. **The framework assumes a formal target exists.** For mathematical activity that is not yet formalisable—conjecture generation, theory construction, taste and judgement—the analysis does not apply. These are exactly the activities that the creativity literature identifies as out of reach [2].

We consider limitation 5 the most important boundary. The framework addresses *reliability of claims already made*, not *generation of good claims*. A system with $\mathcal{V} = 1$ is trustworthy but not necessarily interesting. Conflating the two would be the central error the framework is meant to prevent.

9.3 Relation to the AGI-evaluation discourse

The concern we formalise has a parallel in recent work on measuring general capability. Assessment frameworks that decompose general intelligence into distinct cognitive abilities—and that must therefore define what a given measurement does and does not isolate—face the same attribution problem: a score is only as meaningful as the construct it actually measures. The reliability question we raise for AI4Math is the domain-specific version of this general issue: *what exactly is being certified, and does the certification reach the claim that was made?* We regard this convergence as support for treating construct validity as a first-class design concern rather than a post-hoc statistical worry.

9.4 Future work

Three directions follow. (i) Instrument an existing neuro-symbolic prover to emit SRIR graphs and measure $\mathcal{V}$ on a standard benchmark, testing the narrowing hypothesis. (ii) Extend the taxonomy through systematic annotation of reduction errors in a large corpus of model-generated formalisations, refining D1–D7 into a validated instrument with inter-annotator agreement statistics. (iii) Study whether $\mathcal{V}$ can serve as a training signal, closing the loop between the measurement and the reduction layer that the measure indicts.

10 Conclusion

Reliability in AI for Mathematics has been pursued as a verification problem: add a checker, trust the checker. We have argued that this framing has a structural blind spot. Because a proof assistant certifies the formal claim it is given and is silent about whether that claim is the intended one, the informal-to-formal reduction step can fail undetectably, and no improvement to the checker addresses that failure. We located this blind spot at a specific architectural position—the semantic reduction layer—and gave three instruments for working with it: a typed intermediate representation that makes anchoring explicit, a graded verifiability score that makes partial progress measurable, and a defect taxonomy that gives failures names. We closed with a falsifiable hypothesis and a checklist for practitioners.

The claim we would most like to see tested is the simplest one: that disclosing which steps of a machine-produced proof are anchored, and which are not, changes what we can responsibly conclude from it. If that is right, then the next advance in reliable AI4Math will not be a stronger checker but a more honest reduction layer.

Acknowledgements

This work was conducted within the Elite 20 Program as a study of AI-assisted academic research. The author thanks the authors of the reference materials for making their lecture notes available. All large language model outputs used in preparing this paper were independently

verified against primary sources; the verification record is maintained in the accompanying AI collaboration log.

References

- [1] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.
- [2] Xiaoyang Chen, Xinan Dai, Yu Du, Qian Feng, Naixu Guo, Tingshuo Gu, Xiang Jiang, others, and Ruichong Zhang. Deepmath-creative: A benchmark for evaluating mathematical creativity of large language models, 2025.
- [3] Yuri Chervonyi, Trieu H. Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang H. Nguyen, Marcelo Menegali, Junehyuk Jung, Junsu Kim, Vikas Verma, Quoc V. Le, and Thang Luong. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2. *Journal of Machine Learning Research*, 26(241):1–39, 2025.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.
- [5] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In Amy P. Felty and Aart Middeldorp, editors, *Automated Deduction – CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388, Cham, 2015. Springer.
- [6] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.
- [7] Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, and Julius Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.
- [8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021), Datasets and Benchmarks Track*, 2021.
- [9] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, others, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026.
- [10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023.
- [11] Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover: A frontier model

for open-source automated theorem proving, 2025. [C1] Preprint identifier corrected: the secondary source cited arXiv:2502.07922, an unrelated robotics paper.

- [12] Filip Marić. A survey of interactive theorem proving. In *Logic in Computer Science II*, volume 18 of *Zbornik radova (Collection of Papers)*, pages 173–223. Mathematical Institute SANU, Belgrade, 2015.
- [13] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [14] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP ’20)*, pages 367–381, New York, NY, USA, 2020. ACM.
- [15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837, 2022.
- [16] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- [17] Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in AI, 2024.
- [18] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.
- [19] Jin Peng Zhou, Charles Staats, Wenda Li, Christian Szegedy, Kilian Q. Weinberger, and Yuhuai Wu. Don’t trust: Verify – grounding LLM quantitative reasoning with autoformalization. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

A Reliability Checklist (Full Form)

The checklist below is intended to be applied in order. Each item states the question, the evidence required, and the failure it detects.

Table 2: Full reliability checklist for auditing an AI4Math claim.

#	Question	Evidence required	Detects
1	What was formalised?	Formal statement + informal source	—
2	Is the mapping disclosed?	Step-level correspondence	D1–D6
3	Which steps are anchored?	SRIR graph, $\alpha(\cdot)$	D7
4	Are unanchored steps load-bearing?	Dependency cone	D7
5	Do library citations hold?	Lemma hypotheses vs. usage	D4
6	What is the claim’s boundary?	Domain + coverage assumptions	D5
7	Does the result transfer?	Informal/formal language distance	D1, D6

B A Worked SRIR Fragment

For concreteness we give the SRIR representation of a short derivation, showing how anchoring status is recorded. The example is illustrative and uses a schematic syntax rather than a specific implementation.

Listing 1: SRIR fragment for a two-step derivation. Note that the unanchored node is retained, not dropped, and that its status propagates to the goal.

```
( srir
  ( node n1 : type Assumption : anchor U
    :gloss "n is a natural number greater than 1"
    :formal "n : Nat, h : n > 1"
    :note "hypothesis stated informally but never
           discharged as an explicit premise" )

  ( node n2 : type LemmaRef : anchor A
    :gloss "n has a prime factor by the fundamental theorem"
    :formal "Nat.exists_prime_and_dvd"
    :deps (n1)
    :note "lemma exists in mathlib; hypotheses match" )

  ( node n3 : type Goal : anchor U
    :gloss "therefore n has a prime factor"
    :formal "exists p, Nat.Prime p AND p | n"
    :deps (n2)
    :note "status inherited from cone: n1 unanchored" ) )

; verifiability: V = 1/3 under uniform weighting
; semantic closure: FAILS at n1
; actionable: promote the condition in n1 to an
; explicit premise; re-run anchoring pass
```

The fragment illustrates three properties of the representation. The unanchored assertion is *visible*: it appears as a node with $\alpha = U$ rather than as a fluent sentence in a paragraph. The failure is *localised*: it is attributable to a specific node and a specific defect class. And the remedy is *stated*: the audit output says what would change the verdict.